

Proyecto 3

Universidad del Valle de Guatemala – Bases de Datos 2

Hugo Barillas Pablo Barillas Osman De León José Mérida
Roberto Nájera Milton Polanco

28 de mayo de 2026

Repositorio: [GitHub](#) — Proyecto 3

Índice

1. Descripción del Problema	2
2. Justificación de la Elección de Base de Datos	2
2.1. Tecnología seleccionada: Neo4j	2
2.2. Comparación con alternativas	2
3. Diseño del Modelo de Datos	3
3.1. Convención de etiquetado físico	3
3.2. Nodos	4
3.3. Relaciones	4
3.4. Diagrama del esquema	4
3.5. Escalabilidad del modelo	5
3.6. Unicidad global entre rompecabezas	5
4. Implementación del Modelo	5
5. Algoritmo de Resolución	6
5.1. Descripción	6
5.2. Pseudocódigo	6
5.3. Uso de Neo4j en el algoritmo	7
6. Manejo de Piezas Faltantes	7

1. Descripción del Problema

Se nos entregó un rompecabezas físico con piezas interconectadas mediante *tabs* (picos) y *huecos*. El objetivo es construir un sistema que almacene la estructura del rompecabezas en una base de datos y genere, a partir de una pieza inicial, instrucciones ordenadas de ensamblaje. El sistema debe manejar el caso de piezas faltantes y seguir armando la mayor parte posible del rompecabezas.

Lo importante a notar es que el problema es fundamentalmente *relacional*: no importa la forma ni el color de cada pieza, sino únicamente qué conector de qué pieza encaja con qué conector de otra pieza. Esta observación es la que guía directamente la elección de tecnología.

2. Justificación de la Elección de Base de Datos

2.1. Tecnología seleccionada: Neo4j

Se eligió **Neo4j** (base de datos orientada a grafos) porque refleja directamente la estructura del problema. En un rompecabezas, el dato central no es un atributo de una entidad aislada, sino la *conexión* entre dos conectores de piezas distintas. En Neo4j esa conexión es un arco de primer nivel que existe por sí mismo; en un modelo relacional sería una fila en una tabla de unión que solo cobra sentido al hacer un *JOIN*. Eliminar ese nivel de indirección simplifica tanto el modelo como las consultas.

Adicionalmente, Neo4j ofrece ventajas concretas para este dominio:

- **Traversals nativos.** La pregunta “¿qué pieza se conecta a esta pieza?” es un salto de dos arcos en Cypher y no requiere lógica adicional. En SQL la misma consulta involucraría al menos cuatro tablas y varios *JOIN*.
- **Sin migraciones de esquema.** Añadir un nuevo rompecabezas es insertar un subgrafo aislado; no se modifica ninguna tabla existente ni se corre ningún script de migración.
- **Restricciones de unicidad.** Neo4j soporta restricciones **UNIQUE** equivalentes a claves primarias relacionales, lo que garantiza integridad referencial sin costo de modelado extra.

2.2. Comparación con alternativas

PostgreSQL hubiera sido una opción técnicamente válida, pero introduce fricción innecesaria para este dominio. El modelo requeriría al menos cuatro tablas (**puzzles**, **pieces**, **connectors**, **connections**) y cada consulta de vecindad involucraría múltiples *JOIN*. El BFS en sí requeriría CTEs recursivos, trasladando la lógica de traversal al SQL en lugar de mantenerla limpiamente en la capa de aplicación. Para un problema donde la relación es el dato, un modelo relacional añade indirección sin ningún beneficio.

MongoDB maneja bien documentos anidados, lo que lo hace atractivo a primera vista: una pieza con sus conectores encaja naturalmente en un documento. El problema aparece con las conexiones entre piezas, que son simétricas: el conector A de la pieza 1 y el conector B de la pieza 2 se referencian mutuamente. Representar esa bidireccionalidad en

documentos requiere duplicar la referencia en ambos lados y mantenerlos sincronizados manualmente, introduciendo una fuente de inconsistencia que Neo4j evita por diseño.

DynamoDB hubiera funcionado para rompecabezas muy pequeños con un patrón de acceso simple y bien definido de antemano. Sin embargo, DynamoDB está optimizado para lecturas y escrituras de ítems individuales por clave; las consultas de vecindad que el BFS necesita requieren o bien un diseño de *access patterns* muy cuidadoso con índices secundarios, o bien múltiples roundtrips a la base de datos por cada paso del algoritmo. A medida que el rompecabezas crece, ese costo se vuelve prohibitivo. Para puzzles de juguete de pocas piezas sería suficiente, pero no escala bien con la complejidad del grafo.

3. Diseño del Modelo de Datos

3.1. Convención de etiquetado físico

Antes de cargar el rompecabezas al sistema, cada pieza y cada conector deben etiquetarse físicamente para que el ensamblador pueda identificarlos a mano durante el armado.

- **Piezas:** se escribe un número en la parte delantera de cada pieza para identificarla de forma única dentro del rompecabezas. Se recomienda usar el formato <número>-<nombre> (p.ej. 1 - Cabeza) para que la identificación sea inequívoca incluso cuando varias piezas tienen formas similares.
- **Conectores:** se escribe una letra o código corto con cinta adhesiva junto a cada conector. La etiqueta es *local* a su pieza, de modo que dos piezas distintas pueden tener un conector A sin conflicto.

Con este esquema, un conector queda identificado unívocamente por el par (pieza, etiqueta), p.ej. (1, A), y una conexión física queda registrada como el par de conectores que encajan entre sí.

Las figuras 1 y 2 muestran dos rompecabezas físicos etiquetados siguiendo esta convención.



Figura 1: Rompecabezas *Plesiosaurs*: 10 piezas numeradas y conectores etiquetados con letras (A, B, C...) junto a cada tab o hueco.



Figura 2: Rompecabezas *Excavator*: 10 piezas numeradas y conectores etiquetados con letras (A, B, C, D) junto a cada conector.

3.2. Nodos

```

:Puzzle      id, name, description
:Piece       id (global), local_id, label, puzzle_id
:Connector   id (global), local_id, label, type (tab/hole), piece_local_id, puzzle_id

```

Tener IDs locales y globales nos permite referenciar una pieza de forma única dentro del modelo de datos, pero al mismo tiempo mantener una representación corta y legible para presentar al usuario. El ID global se construye concatenando el slug del puzzle con el ID local (ver §3.6), y es el que se almacena en Neo4j; el ID local es el que aparece en las instrucciones de ensamblaje.

3.3. Relaciones

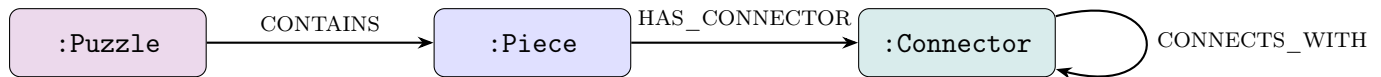
```

CONTAINS      (:Puzzle) → (:Piece)
HAS_CONNECTOR (:Piece) → (:Connector)
CONNECTS_WITH (:Connector) ↔ (:Connector) — almacenado en ambas direcciones

```

Mediante estas relaciones podemos identificar las piezas que pertenecen a cada rompecabezas y todas las conexiones entre ellas. La relación `CONNECTS_WITH` se almacena en ambas direcciones deliberadamente: así cualquier traversal de vecindad puede hacerse con un único patrón `MATCH` sin necesidad de manejar dirección, lo que simplifica notablemente las consultas del algoritmo.

3.4. Diagrama del esquema



3.5. Escalabilidad del modelo

El modelo escala bien en ambas dimensiones relevantes. Agregar un nuevo rompecabezas es insertar un subgrafo completamente aislado: no afecta los nodos ni las relaciones de puzzles existentes, no requiere modificar el esquema y no introduce ningún tipo de contención. Agregar piezas o conectores a un puzzle existente sigue el mismo patrón: nodos nuevos enlazados al subgrafo correspondiente. El namespacing por `puzzle_id` garantiza que el crecimiento horizontal entre puzzles nunca genere colisiones, y las restricciones `UNIQUE` mantienen la integridad sin degradación de rendimiento gracias a los índices que Neo4j crea automáticamente al definirlas.

3.6. Unicidad global entre rompecabezas

Neo4j comparte un único grafo de propiedades entre todos los rompecabezas almacenados. Dado que los IDs de piezas y conectores son locales a su puzzle (dos puzzles distintos pueden tener una pieza P01), es necesario garantizar unicidad global. Para ello, los IDs de `:Piece` y `:Connector` se construyen concatenando el slug del puzzle con el ID local:

$$\text{id} = \langle \text{puzzle_id} \rangle _ \langle \text{local_id} \rangle$$

Se crean tres restricciones `UNIQUE` (`Puzzle.id`, `Piece.id`, `Connector.id`) al inicializar la base de datos, de forma que un intento de insertar dos nodos con el mismo ID global falle a nivel de base de datos antes de llegar a la lógica de aplicación.

4. Implementación del Modelo

La carga de datos sigue un orden determinado: primero el nodo `:Puzzle`, luego cada `:Piece` enlazada con `CONTAINS`, luego cada `:Connector` enlazado con `HAS_CONNECTOR`, y finalmente las relaciones `CONNECTS_WITH` entre pares de conectores. Se utiliza `MERGE` en lugar de `CREATE` para que la operación sea idempotente: ejecutar el seed dos veces no duplica datos.

Listing 1: Inserción de un rompecabezas en Neo4j

```

1  -- Nodo Puzzle
2  MERGE (puz:Puzzle {id: $pid})
3  SET puz.name = $name, puz.description = $desc;
4
5  -- Nodo Piece
6  MATCH (puz:Puzzle {id: $pid})
7  MERGE (pc:Piece {id: $gid})
8  SET pc.local_id=$lid, pc.label=$label, pc.puzzle_id=$pid
9  MERGE (puz)-[:CONTAINS]->(pc);
10
11 -- Nodo Connector

```

```

12 MATCH (pc:Piece {id: $piece_gid})
13 MERGE (c:Connector {id: $gid})
14 SET c.local_id=$lid, c.label=$label, c.type=$type,
15     c.piece_local_id=$plid, c.puzzle_id=$pid
16 MERGE (pc)-[:HAS_CONNECTOR]->(c);
17
18 -- Relacion CONNECTS_WITH (bidireccional)
19 MATCH (ca:Connector {id: $ga}), (cb:Connector {id: $gb})
20 MERGE (ca)-[:CONNECTS_WITH]->(cb)
21 MERGE (cb)-[:CONNECTS_WITH]->(ca);

```

El rompecabezas de prueba cargado es un **Plesiosaurio** con 10 piezas (P01–P10), conectores etiquetados A–J y 10 conexiones que forman la columna vertebral y extremidades del animal. Este puzzle sirve para validar tanto el caso completo (todas las piezas disponibles) como el caso parcial (una o más piezas marcadas como faltantes).

5. Algoritmo de Resolución

5.1. Descripción

El algoritmo aplica **BFS (Breadth First Search)** sobre el grafo de piezas, expandiendo la frontera conector por conector a partir de una pieza inicial elegida por el usuario. Se elige BFS sobre DFS porque produce instrucciones en orden de distancia desde la pieza inicial, lo que resulta más natural para un ensamblador físico: primero se colocan las piezas directamente adyacentes, luego las que están a dos pasos, y así sucesivamente.

Entrada Grafo $G = (P, C, E)$, pieza inicial $s \in P$, conjunto de piezas faltantes $M \subseteq P$.

Salida Lista ordenada de pasos de ensamblaje, cada uno con estado **start**, **placed** o **missing**.

5.2. Pseudocódigo

Listing 2: BFS de ensamblaje

```

1 visited = {s}; queue = [s]
2 emitir paso START: "Coloca la pieza s como punto de partida"
3
4 mientras queue no este vacio:
5     u = queue.pop_left()
6     para cada conector  $c_u$  de u:
7          $c_v$  = vecino( $c_u$ ) // CONNECTS_WITH en Neo4j
8         v = pieza( $c_v$ )
9         si v en M (faltante) y no reportada:
10             emitir paso MISSING
11             continuar
12         si v en visited: continuar
13         visited.add(v); queue.append(v)
14         emitir paso PLACED:
15             "Toma la pieza v, conecta su  $c_v$ .tipo ' $c_v$ .label'
16             al  $c_u$ .tipo ' $c_u$ .label' de la pieza u"

```

```

17
18 para cada p en P - visited - M:
19     emitir paso MISSING: "pieza p disponible pero inalcanzable"

```

La complejidad del algoritmo es $\mathcal{O}(n + m)$ donde $n = |P|$ es el número de piezas y $m = |E|$ el número de conexiones. Cada pieza se visita a lo sumo una vez y cada conexión se examina a lo sumo dos veces (una desde cada extremo), por lo que el algoritmo escala linealmente con el tamaño del rompecabezas.

5.3. Uso de Neo4j en el algoritmo

Al cargar el rompecabezas, se ejecuta una única consulta Cypher que materializa el mapa completo de vecinos ($\text{connector_id} \rightarrow \text{connector_id}$) en memoria:

```

1 MATCH (:Puzzle {id: $pid})-[:CONTAINS]->(:Piece)
2     -[:HAS_CONNECTOR]->(ca:Connector)
3     -[:CONNECTS_WITH]->(cb:Connector)
4 WHERE ca.puzzle_id = $pid AND cb.puzzle_id = $pid
5     AND ca.local_id < cb.local_id
6 RETURN ca.local_id AS a, cb.local_id AS b;

```

El BFS opera sobre este mapa en memoria; la base de datos provee la representación persistente y compartida. Dado que la cantidad de datos por rompecabezas es pequeña (decenas o cientos de piezas), cargar el grafo completo de una sola vez es más eficiente que consultar Neo4j por cada paso del BFS, evitando la latencia de red en cada iteración.

6. Manejo de Piezas Faltantes

Una de las funcionalidades clave del sistema es continuar generando instrucciones útiles incluso cuando algunas piezas no están disponibles. Cuando durante el BFS se encuentra una pieza $v \in M$:

1. Se registra un paso MISSING indicando qué conector debería unirse y de qué pieza, para que el ensamblador sepa qué falta.
2. No se encola v , bloqueando esa rama de expansión y evitando instrucciones que no se pueden ejecutar.
3. Las piezas alcanzables por rutas alternativas (que no pasen por v) se ensamblan con normalidad, aprovechando al máximo las piezas disponibles.
4. Al finalizar, las piezas disponibles pero inalcanzables debido al bloqueo se reportan explícitamente, dando al ensamblador una imagen completa del estado del rompecabezas.

Esto refleja el comportamiento físico deseado: el ensamblador arma todo lo posible, recibe una notificación clara de qué sección no puede completarse y por qué, y puede retomar el armado si la pieza faltante aparece posteriormente.